Zero Downtime Database Deployment Strategies

A comprehensive guide with step-by-step examples

Overview

Zero-downtime database deployment ensures your application remains fully available while schema changes, data migrations, or database upgrades are applied. The fundamental principle is: **the database must always be compatible with both the currently running version of the app and the version being deployed.**

1. Expand / Contract Pattern

The safest approach for schema changes, done in 3 separate deployments. **Scenario:** Rename column `user_name` to `full_name` in a users table.

Phase 1 - Expand (backward-compatible, old app still works)

```
-- Add new column alongside old one
ALTER TABLE users ADD COLUMN full_name VARCHAR(255);

-- Backfill existing data
UPDATE users SET full_name = user_name WHERE full_name IS NULL;
```

Phase 2 - Migrate app (dual-write to both columns)

```
// App writes to BOTH columns
user.setUserName(name);    // old
user.setFullName(name);    // new

// Reads still from old column
return user.getUserName();
```

Phase 3 - Switch reads to new column

```
// Now read from new column
return user.getFullName();
// Still writing to both as safety net
```

Phase 4 - Contract (separate deployment, after confidence)

```
ALTER TABLE users DROP COLUMN user_name;
```

---

2. Blue / Green Database

Scenario: Upgrade Aurora PostgreSQL 13 to 15. Spin up a new database alongside the old, sync data via replication, then cut over at the connection string level.

Step 1 - Create Green from snapshot

```
aws rds create-db-cluster \
  --db-cluster-identifier myapp-green \
  --engine aurora-postgresql \
  --engine-version 15.3 \
  --snapshot-identifier myapp-blue-snapshot
```

Step 2 - Enable logical replication Blue to Green

```
-- On Blue (source)
CREATE PUBLICATION blue_pub FOR ALL TABLES;

-- On Green (target)
CREATE SUBSCRIPTION green_sub
  CONNECTION 'host=blue-db.xxx.rds.amazonaws.com dbname=myapp'
  PUBLICATION blue_pub;
```

Step 3 - Verify Green is caught up

```
SELECT now() - pg_last_xact_replay_timestamp() AS replication_lag;
-- Should be < 1 second before cutover
```

Step 4 - Cutover (seconds of downtime at most)

```
aws secretsmanager update-secret \
  --secret-id myapp/db-url \
  --secret-string '{"host":"myapp-green.xxx.rds.amazonaws.com"}'

kubectl rollout restart deployment/myapp
```

Step 5 - Rollback if needed

```
aws secretsmanager update-secret \
  --secret-id myapp/db-url \
  --secret-string '{"host":"myapp-blue.xxx.rds.amazonaws.com"}'
```

---

3. Dual Writes + Feature Flags

Scenario: Migrating orders from `orders` table to `orders_v2` with a new `currency` field.

Step 1 - Create new table

```
CREATE TABLE orders_v2 (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id  INT,
  total_amount DECIMAL(10,2),
  currency     CHAR(3),        -- new field
  created_at   TIMESTAMPTZ
);
```

Step 2 - Dual-write in application code

```
def create_order(customer_id, amount, currency):
    # Always write to old table
    old_order = db.execute("""
        INSERT INTO orders (customer_id, total) VALUES (%s, %s) RETURNING id
    """, [customer_id, amount])

    # Write to new table if feature flag enabled
    if feature_flags.is_enabled('use_orders_v2'):
        db.execute("""
            INSERT INTO orders_v2 (customer_id, total_amount, currency)
            VALUES (%s, %s, %s)
        """, [customer_id, amount, currency])
    return old_order
```

Step 3 - Backfill historical data (background job)

```
def backfill_orders():
    batch_size, last_id = 1000, 0
    while True:
        rows = db.execute("""
            SELECT id, customer_id, total FROM orders
            WHERE id > %s ORDER BY id LIMIT %s
```

```
        """, [last_id, batch_size])
        if not rows: break
        db.execute_many("""
            INSERT INTO orders_v2 (customer_id, total_amount, currency)
            VALUES (%s, %s, 'GBP') ON CONFLICT DO NOTHING
        """, [(r.customer_id, r.total) for r in rows])
        last_id = rows[-1].id
        time.sleep(0.1)  # throttle
```

Step 4 - Shift reads gradually, then drop old table

```
# Increase rollout % from 0 to 100 via feature flag
if feature_flags.get_rollout('read_orders_v2') > random():
    return db.execute('SELECT * FROM orders_v2 WHERE customer_id = %s', [id])

# Once at 100%:
DROP TABLE orders;
ALTER TABLE orders_v2 RENAME TO orders;
```

---

4. Online Schema Change with gh-ost

Scenario: Add an index to a 50M row `transactions` table without locking it.

Without gh-ost (dangerous)

X CREATE INDEX idx_transactions_user ON transactions(user_id); -- Locks the entire table for minutes/hours on large tables

With gh-ost (safe)

```
gh-ost \
  --user='admin' --password='secret' \
  --host='mydb.rds.amazonaws.com' \
  --database='myapp' --table='transactions' \
  --alter='ADD INDEX idx_transactions_user (user_id)' \
  --allow-on-master --execute
```

What gh-ost does internally

- Creates shadow table: `_transactions_gho`
- Copies rows in small batches (1000 rows at a time)
- Tails binlog to capture live writes during copy
- Applies live changes to shadow table in parallel
- Atomically renames: _transactions_gho to transactions

Monitor progress

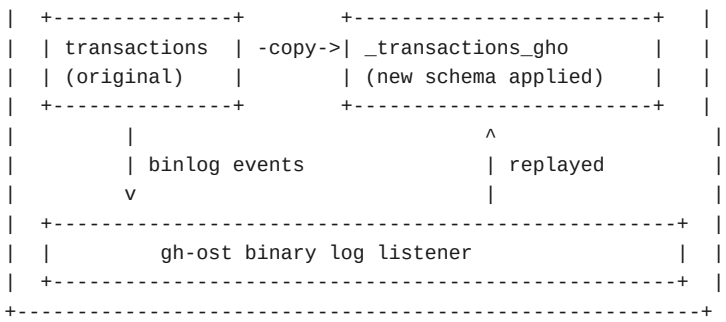
```
echo "status" | nc -U /tmp/gh-ost.myapp.transactions.sock
# Migrating 50,234,891 rows. 34% complete. ETA: 12m
```

4b. gh-ost - Elaborated Approach

gh-ost (GitHub's Online Schema Transmogrifier) is a **triggerless** online schema change tool for MySQL/Aurora MySQL. Unlike pt-online-schema-change which uses triggers, gh-ost tails the binary log, making it far safer under heavy write load.

How it Works Internally

```
+-----------------------------------------------------+
|                  Your Production DB                 |
|                                                     |
```



gh-ost never touches the original table with triggers. It: (1) creates a ghost table with the new schema, (2) copies rows in small batches with throttling, (3) simultaneously tails the binlog and applies live DML to the ghost table, and (4) once the ghost catches up, does an atomic table rename.

Complete Real-World Example

Scenario: Add an index + a new nullable column to a 200M row `transactions` table, with zero locking.

Step 1 - Baseline check

```
-- Check current schema
SHOW CREATE TABLE transactions\G

-- Check table size
SELECT
  table_rows,
  ROUND(data_length / 1024 / 1024 / 1024, 2) AS data_gb,
  ROUND(index_length / 1024 / 1024 / 1024, 2) AS index_gb
FROM information_schema.tables
WHERE table_schema = 'myapp' AND table_name = 'transactions';
```

Step 2 - Dry run first (no changes made)

```
gh-ost \
  --user='ghuser' --password='ghpass' \
  --host='mydb.cluster.rds.amazonaws.com' \
  --database='myapp' --table='transactions' \
  --alter='
    ADD COLUMN processed_at DATETIME(3) NULL,
    ADD INDEX idx_transactions_user_status (user_id, status)
  ' \
  --allow-on-master \
  --dry-run
# Validates connectivity, privileges, binlog format -- makes zero changes
```

Step 3 - Real run with throttle controls

```
gh-ost \
  --user='ghuser' --password='ghpass' \
  --host='mydb.cluster.rds.amazonaws.com' \
  --database='myapp' --table='transactions' \
  --alter='
    ADD COLUMN processed_at DATETIME(3) NULL,
    ADD INDEX idx_transactions_user_status (user_id, status)
  ' \
  --allow-on-master \
  --chunk-size=1000 \
  --max-load='Threads_running=50' \
  --critical-load='Threads_running=100' \
  --max-lag-millis=1500 \
  --throttle-control-replicas='replica1.rds.amazonaws.com' \
  --initially-drop-ghost-table \
  --initially-drop-old-table \
  --ok-to-drop-table \
```

```
  --serve-socket-file=/tmp/gh-ost.myapp.transactions.sock \
  --panic-flag-file=/tmp/gh-ost.panic \
  --execute
```

Step 4 - Monitor in a separate terminal

```
echo "status" | nc -U /tmp/gh-ost.myapp.transactions.sock

# Example output:
# Migrating myapp.transactions; Ghost table is myapp._transactions_gho
# Migrated 87,234,100 out of 200,000,000 rows; 43.6% done
# State: migrating; ETA: 28m32s
# Chunk: 1000; Lag: 12ms; Load: Threads_running=18

# Throttle manually (pause copy without stopping)
echo "throttle" | nc -U /tmp/gh-ost.myapp.transactions.sock

# Resume
echo "no-throttle" | nc -U /tmp/gh-ost.myapp.transactions.sock

# Change chunk size on the fly
echo "chunk-size=500" | nc -U /tmp/gh-ost.myapp.transactions.sock
```

Step 5 - Emergency abort

```
# Safe abort -- gh-ost cleans up the ghost table
touch /tmp/gh-ost.panic

# Or via socket
echo "panic" | nc -U /tmp/gh-ost.myapp.transactions.sock
```

The Atomic Rename (Final Step)

When gh-ost is fully caught up, it executes this internally, taking a lock only for milliseconds:

```
-- gh-ost does this internally
LOCK TABLES transactions WRITE, _transactions_gho WRITE;

RENAME TABLE
  transactions       TO _transactions_del,
  _transactions_gho  TO transactions;

UNLOCK TABLES;
-- _transactions_del is dropped afterward (if --ok-to-drop-table was passed)
```

Privileges Required

```
CREATE USER 'ghuser'@'%' IDENTIFIED BY 'ghpass';

GRANT ALTER, CREATE, DELETE, DROP, INDEX, INSERT,
      LOCK TABLES, SELECT, SUPER, TRIGGER, UPDATE
ON myapp.*
TO 'ghuser'@'%';

-- Also needs replication access for binlog tailing
GRANT REPLICATION CLIENT, REPLICATION SLAVE ON *.* TO 'ghuser'@'%';
```

DB Config Prerequisites

```
-- Must be ROW format (not STATEMENT)
SET GLOBAL binlog_format = 'ROW';

-- Verify
SHOW GLOBAL VARIABLES LIKE 'binlog_format';  -- should be ROW
SHOW GLOBAL VARIABLES LIKE 'log_bin';         -- should be ON
```

gh-ost vs pt-online-schema-change

gh-ost tracks writes via binlog tailing; **pt-osc** uses triggers on the original table. Under high write load, pt-osc triggers add latency to every INSERT/UPDATE/DELETE. gh-ost's async binlog replay avoids this entirely. gh-ost also supports pause/resume via socket, has built-in replica lag throttling, and rollback is simply dropping the ghost table. pt-osc rollback requires manually removing triggers, which is riskier under load.

Common Failure Scenarios

Binlog not in ROW format

```
FATAL binlog_format is not ROW
Fix: SET GLOBAL binlog_format = 'ROW';
```

Previous ghost table left behind

```
FATAL Found ghost table _transactions_gho
Fix: Pass --initially-drop-ghost-table
```

Replica lag too high

```
INFO Throttling due to replication lag: 2340ms > 1500ms
# gh-ost auto-pauses and resumes -- expected behavior, not a failure
```

Insufficient privileges

```
FATAL Need SUPER or REPLICATION CLIENT privilege
Fix: GRANT REPLICATION CLIENT, REPLICATION SLAVE ON *.* TO 'ghuser'@'%';
```

---

5. Read Replica Promotion

Scenario: Major restructuring -- splitting `products` into `products` + `product_pricing`.

Step 1 - Create read replica

```
aws rds create-db-instance-read-replica \
  --db-instance-identifier myapp-replica \
  --source-db-instance-identifier myapp-primary
```

Step 2 - Stop replication, apply migration on replica

```
CALL mysql.rds_stop_replication;

CREATE TABLE product_pricing AS
  SELECT id, price, currency, updated_at FROM products;

ALTER TABLE products DROP COLUMN price, DROP COLUMN currency;
ALTER TABLE product_pricing ADD PRIMARY KEY (id);
```

Step 3 - Promote replica, switch connection string

```
aws rds promote-read-replica \
  --db-instance-identifier myapp-replica

# Update DNS / Secrets Manager to new primary
aws route53 change-resource-record-sets \
  --hosted-zone-id ZXXX \
  --change-batch '{"Changes":[{"Action":"UPSERT",
    "ResourceRecordSet":{"Name":"db.myapp.internal",
    "Type":"CNAME","TTL":30,
    "ResourceRecords":[{"Value":"myapp-replica.xxx.rds.amazonaws.com"}]}}]}'
```

---

6. Changing a Column Datatype

Most databases rewrite the entire table for datatype changes, causing a full lock. Use the expand/contract approach with a trigger to sync writes.

Safe Steps -- INT to BIGINT (Scenario: order_id running out of int range)

Step 1 - Add new column

```
ALTER TABLE orders ADD COLUMN order_id_new BIGINT;
```

Step 2 - Backfill in batches

```
DO $$
DECLARE batch_start INT := 0; batch_size INT := 10000;
BEGIN
  LOOP
    UPDATE orders
    SET order_id_new = order_id::BIGINT
    WHERE id > batch_start AND id <= batch_start + batch_size
      AND order_id_new IS NULL;
    EXIT WHEN NOT FOUND;
    batch_start := batch_start + batch_size;
    PERFORM pg_sleep(0.05);
  END LOOP;
END $$;
```

Step 3 - Trigger to sync live writes

```
CREATE OR REPLACE FUNCTION sync_order_id()
RETURNS TRIGGER AS $$
BEGIN
  NEW.order_id_new := NEW.order_id::BIGINT;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sync_order_id
BEFORE INSERT OR UPDATE ON orders
FOR EACH ROW EXECUTE FUNCTION sync_order_id();
```

Step 4 - Verify, index, then atomic rename

```
-- Verify no nulls
SELECT COUNT(*) FROM orders WHERE order_id_new IS NULL;  -- must be 0

-- Non-blocking index
CREATE INDEX CONCURRENTLY idx_orders_order_id_new ON orders(order_id_new);

-- Atomic rename
BEGIN;
  ALTER TABLE orders RENAME COLUMN order_id TO order_id_old;
  ALTER TABLE orders RENAME COLUMN order_id_new TO order_id;
COMMIT;
```

Step 5 - Cleanup (next deployment)

```
DROP TRIGGER trg_sync_order_id ON orders;
DROP FUNCTION sync_order_id();
ALTER TABLE orders DROP COLUMN order_id_old;
```

---

7. Changing Numeric to VARCHAR

A semantic type change -- one-way, so treat carefully. **Scenario:** `products.price` from `DECIMAL(10,2)` to `VARCHAR(20)` to support values like `POA`, `TBC`.

Step 1 - Add VARCHAR column

```
ALTER TABLE products ADD COLUMN price_text VARCHAR(20);
```

Step 2 - Backfill: convert existing numeric values

```
DO $$
DECLARE batch INT := 0;
BEGIN
  LOOP
    UPDATE products
    SET price_text = price::TEXT
    WHERE id > batch AND id <= batch + 10000
      AND price_text IS NULL;
    EXIT WHEN NOT FOUND;
    batch := batch + 10000;
    PERFORM pg_sleep(0.05);
  END LOOP;
END $$;
```

Step 3 - Trigger to sync live writes

```
CREATE OR REPLACE FUNCTION sync_price_text()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.price IS NOT NULL THEN
    NEW.price_text := NEW.price::TEXT;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sync_price_text
BEFORE INSERT OR UPDATE ON products
FOR EACH ROW EXECUTE FUNCTION sync_price_text();
```

Step 4 - Verify consistency

```
-- No nulls remaining
SELECT COUNT(*) FROM products WHERE price_text IS NULL;  -- must be 0

-- Spot-check conversion accuracy
SELECT price, price_text FROM products
WHERE price::TEXT != price_text LIMIT 10;  -- must return 0 rows
```

Step 5 - Atomic rename

```
BEGIN;
  ALTER TABLE products RENAME COLUMN price TO price_numeric_old;
  ALTER TABLE products RENAME COLUMN price_text TO price;
COMMIT;
```

Step 6 - Cleanup

```
DROP TRIGGER trg_sync_price_text ON products;
DROP FUNCTION sync_price_text();
ALTER TABLE products DROP COLUMN price_numeric_old;
```

> ***Important:*** *Once you convert to VARCHAR and users store non-numeric values like 'POA', rollback becomes impossible without data loss. Run dual columns in parallel for at least 2 weeks before dropping the old column.*

Key Rules and Common Pitfalls

Never do these in a single deployment

X ALTER TABLE users RENAME COLUMN user_name TO full_name; -- Old app instances still running will immediately break

X ALTER TABLE orders ADD COLUMN status VARCHAR NOT NULL; -- Rewrites entire table on Postgres < 11, causes full lock

Always do this instead

```
-- Step 1: Add nullable
ALTER TABLE orders ADD COLUMN status VARCHAR;

-- Step 2: Backfill
UPDATE orders SET status = 'pending' WHERE status IS NULL;

-- Step 3: Add constraint
ALTER TABLE orders ALTER COLUMN status SET NOT NULL;
```

Decision Table

Small table (<1M rows): ALTER TABLE directly, accept brief lock.

MySQL/Aurora MySQL large table: Use gh-ost.

Postgres -- increasing VARCHAR length or INT to BIGINT (Pg 14+): ALTER TABLE directly (no rewrite).

Postgres -- all other type changes: Expand/Contract with new column + trigger + rename.

Major version upgrade: Blue/Green or Read Replica Promotion.

Complex data migration: Dual writes + Feature Flags.

> ***The golden rule:*** *Never let a schema change and a code change that depends on it deploy at the same time. Always deploy the schema change first, verify, then deploy the code change.*